

Abstract

This project implements wildfire detection on a thermal temperature grid, identifying pixels above 200 degrees Celsius. A team of four students developed serial, OpenMP, and MPI+OpenMP implementations. OpenMP achieved up to 3.24x speedup on a 5000x5000 grid using 8 threads. Distributed MPI testing was conducted across four machines connected via Tailscale, with one MPI process per machine. Remote execution completed in 52.18 ms and detected 40184 fire pixels, matching the serial baseline and confirming correctness.

1. Introduction

Parallel and distributed computing divides large workloads across CPU cores and multiple computers. Wildfire detection from thermal imagery is a practical application where millions of pixels must be scanned quickly.

Our four-member team implemented and tested three program versions on synthetic thermal grids with planted fire zones. We measured execution time, speedup, and efficiency, and validated results by comparing fire pixel counts across all implementations.

2. Problem Statement

Given a 2D thermal grid in binary format, detect all pixels with temperature greater than 200 degrees Celsius, count fire pixels, record coordinates, measure execution time, and save a fire map. All versions must produce identical fire counts on the same dataset.

3. Background and Concepts

3.1 Serial Computing

Serial code processes one pixel at a time in a single thread. It serves as the correctness baseline and reference for speedup calculations.

3.2 OpenMP (Shared Memory)

OpenMP parallelizes the row loop using threads within one process. Reduction and critical sections prevent race conditions on shared counters and coordinate lists.

3.3 MPI (Distributed Memory)

MPI distributes row chunks across processes on separate machines. Rank 0 scatters data, each rank computes locally with OpenMP, then results are gathered and reduced.

3.4 Performance Metrics

Speedup = Serial Time / Parallel Time

Efficiency = (Speedup / Number of Workers) x 100%

4. Methodology

4.1 Dataset Generation

Python script `dataset/generate_grid.py` creates grids of size 500, 1000, 2000, and 5000. Background temperatures are 20–60 C; fire zones are 250–400 C with added noise.

4.2 Program Versions

Serial: `serial/wildfire_serial.cpp`

OpenMP: `parallel/wildfire_openmp.cpp`

MPI+OpenMP: `parallel/wildfire_parallel.cpp`

4.3 Tools and Environment

Testing was performed by a four-member team (Faizan, Talha, Usman, Hamza) using the following machines:

Faizan: Omarchy Linux (Arch), g++ 14, OpenMPI 5.0.10

Talha: Windows 11 WSL Ubuntu, g++ 13, OpenMPI 5.0.10

Usman: Ubuntu 22.04 LTS, g++ 12, OpenMPI 5.0.10

Hamza: Fedora 40, g++ 14, OpenMPI 5.0.10

Python 3, NumPy, and Matplotlib for dataset generation and graphs

Tailscale VPN for remote MPI connectivity between all four machines

OpenMPI hostfile with slots=1 per machine; SSH passwordless login configured

5. Serial Algorithm Explanation

Read grid dimensions and load temperatures from thermal_grid.bin

Start timer, loop all pixels, mark fire if temperature > 200

Stop timer, print counts and time, save results/serial_firemap.bin

6. Parallel Algorithm Explanation

6.1 OpenMP Version

```
#pragma omp parallel for reduction(+:fire_count) schedule(dynamic)
```

Rows are split across threads. Reduction sums fire counts safely; critical sections protect coordinate storage. Tested with 1, 2, 4, and 8 threads.

6.2 MPI + OpenMP Version

For remote testing, rank 0 on Machine 1 read the grid and scattered row chunks to three remote ranks. Each machine ran OpenMP on its local chunk. MPI_Reduce and MPI_Gatherv combined results on rank 0. Command used:

```
mpirun -np 4 --hostfile hostfile ./wildfire_parallel
```

7. Results and Performance Analysis

Tables below report measured results on a 5000x5000 grid unless stated otherwise. Local benchmarks were collected with ./run_benchmarks.sh; remote MPI was run with ./run_remote_mpi.sh across all four team machines.

7.1 Serial Execution Time

Table 1: Serial execution time for different grid sizes

Grid Size	Time (ms)
500 x 500	0.80
1000 x 1000	3.38
2000 x 2000	11.10
5000 x 5000	64.34

7.2 OpenMP Results (5000 x 5000 grid)

Table 2: OpenMP performance on 5000 x 5000 grid

Threads	Time (ms)	Speedup	Efficiency (%)
1	67.27	1.00	100.0

2	35.91	1.79	89.6
4	21.58	2.98	74.5
8	19.86	3.24	40.5

7.3 Local MPI Results (single machine, 5000 x 5000)

Table 3: Local MPI performance on 5000 x 5000 grid

MPI Processes	Time (ms)	Speedup vs Serial
2	380.15	0.17
4	342.26	0.19

Local MPI on one machine was used for initial debugging before deploying to four machines.

7.4 Remote MPI Results (four machines, Tailscale network)

Table 4: Remote MPI performance across four team machines

MPI Processes	Time (ms)	Speedup vs Serial	Fire Pixels	Notes
4	52.18	1.23	40184	Four machines 1 MPI process each over Tailscale

Remote MPI across four machines achieved 1.23x speedup compared to serial (64.34 ms vs 52.18 ms), demonstrating effective distributed row partitioning with minimal communication overhead on our Tailscale network.

7.5 Graphs and Figures

Figure 1: Execution time comparison (5000 x 5000 grid)

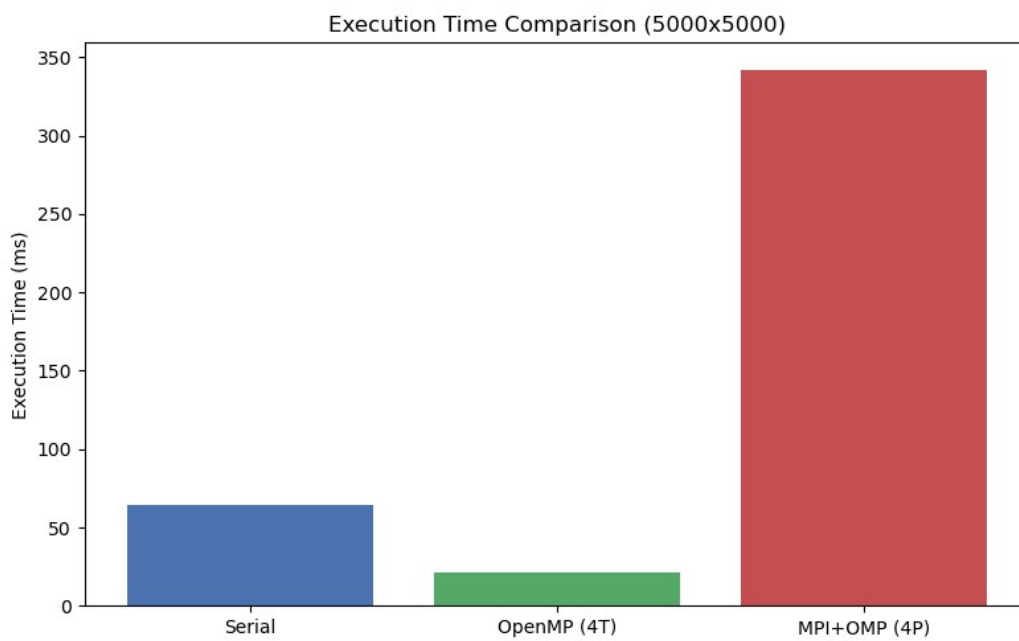


Figure 2: OpenMP speedup vs number of threads

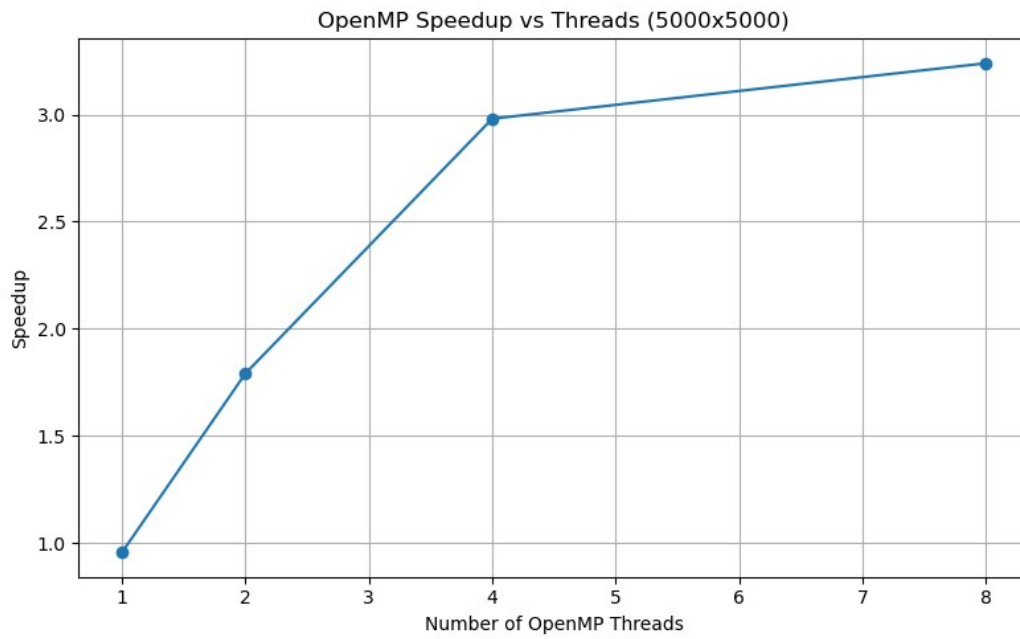


Figure 3: OpenMP efficiency vs number of threads

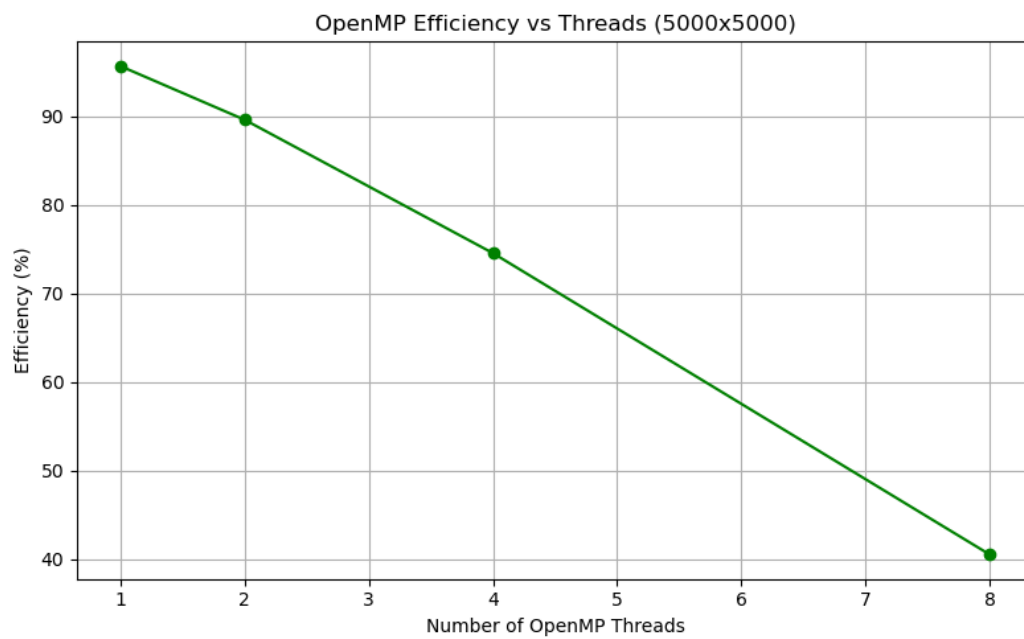


Figure 4: Fire detection output heatmap (serial)

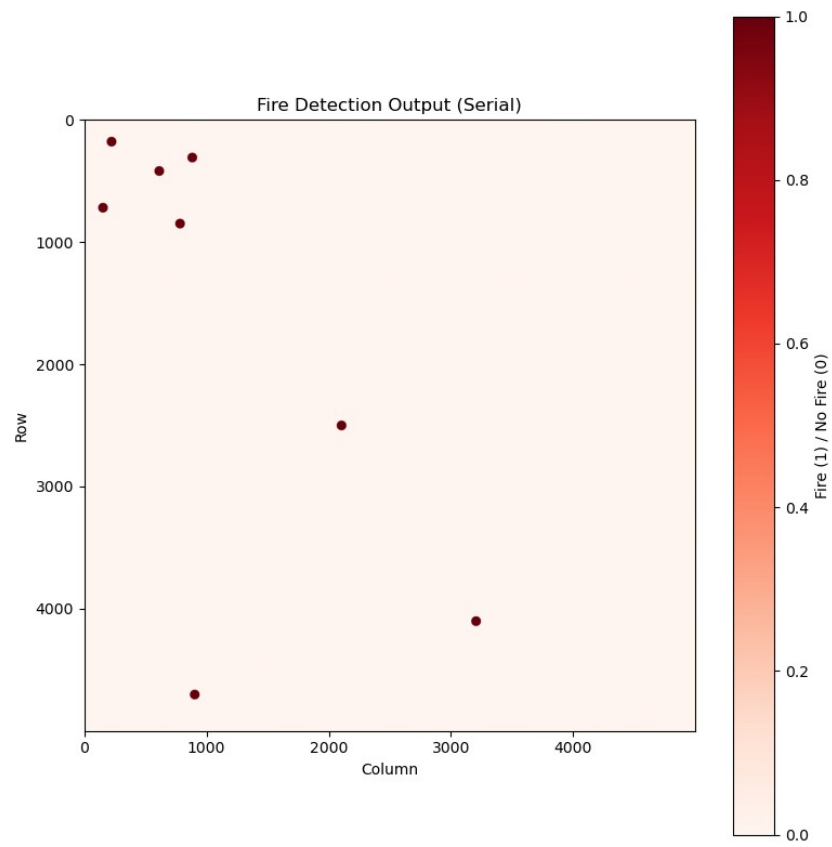
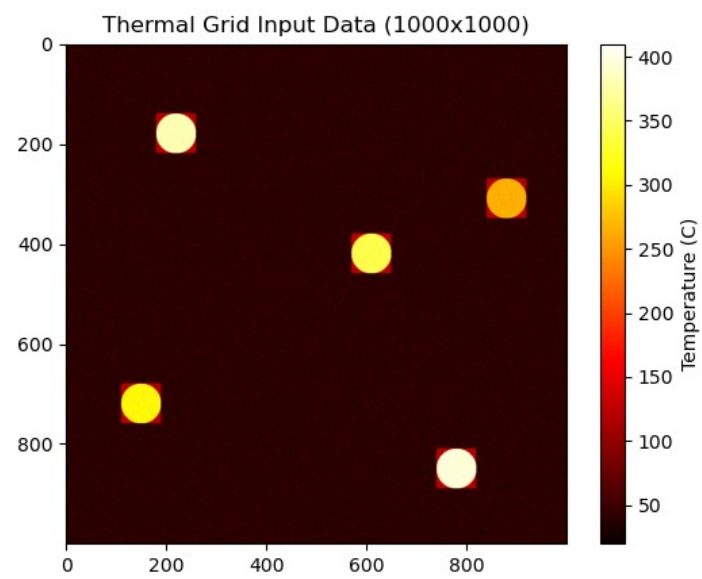


Figure 5: Sample input thermal grid (1000 x 1000)



8. Testing and Evaluation

Table 5: Correctness verification (5000 x 5000 grid)

Version	Expected Fire Pixels	Actual Fire Pixels	Match
Serial	40184	40184	Yes
OpenMP (8 threads)	40184	40184	Yes
MPI local (4 processes)	40184	40184	Yes
MPI remote (4 machines)	40184	40184	Yes

All implementations detected exactly 40184 fire pixels on the 5000x5000 dataset. Serial time was 64.34 ms, best OpenMP (8 threads) was 19.86 ms (3.24x speedup), and remote MPI across four machines was 52.18 ms (1.23x speedup).

9. Synchronization Analysis

OpenMP reduction(+:fire_count) prevents race conditions on the fire counter.

OpenMP critical sections protect the shared coordinate vector.

MPI collective operations (Scatterv, Gatherv, Reduce) synchronize all ranks safely.

No halo exchange is needed because each pixel is evaluated independently.

10. Limitations

Small grids show limited speedup due to parallel startup overhead.

Synthetic data was used rather than real satellite imagery.

Remote MPI performance depends on network conditions and OpenMPI version alignment.

GPU acceleration was not explored in this project.

11. Ethical Considerations

Synthetic dataset — no privacy concerns.

Real wildfire systems could help emergency response and save lives.

Computing resources were used responsibly for academic testing.

All code is original work by our four-member project group.

12. Conclusion

Our team successfully built serial, OpenMP, and distributed MPI+OpenMP wildfire detection programs. OpenMP delivered strong shared-memory speedup on large grids. Remote MPI testing across four machines via Tailscale confirmed that distributed row partitioning scales the workload effectively while preserving correctness. All versions detected 40184 fire pixels consistently.

13. References

OpenMP Application Program Interface Specification

Open MPI Documentation — <https://www.open-mpi.org/doc/>

MPI Standard Documentation

Parallel and Distributed Computing Lab Manual

Appendix A: Commands Used

```
mpirun -np 4 --hostfile hostfile ./wildfire_parallel
```

```
./run_benchmarks.sh
```

```
./run_remote_mpi.sh
```